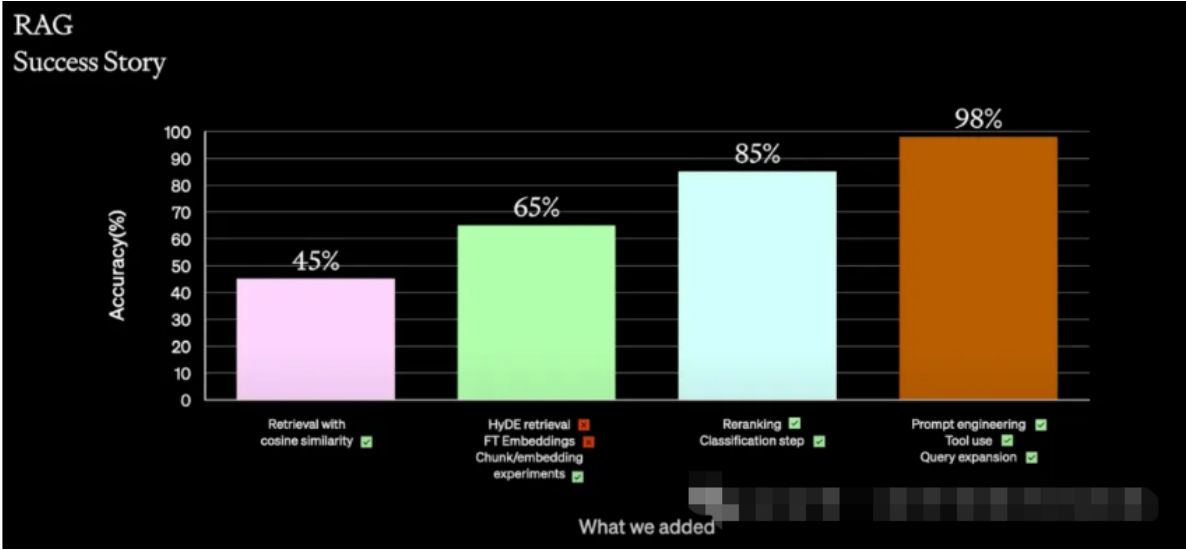


使用RAG技术构建企业级文档问答系统之QA抽取

一、概述

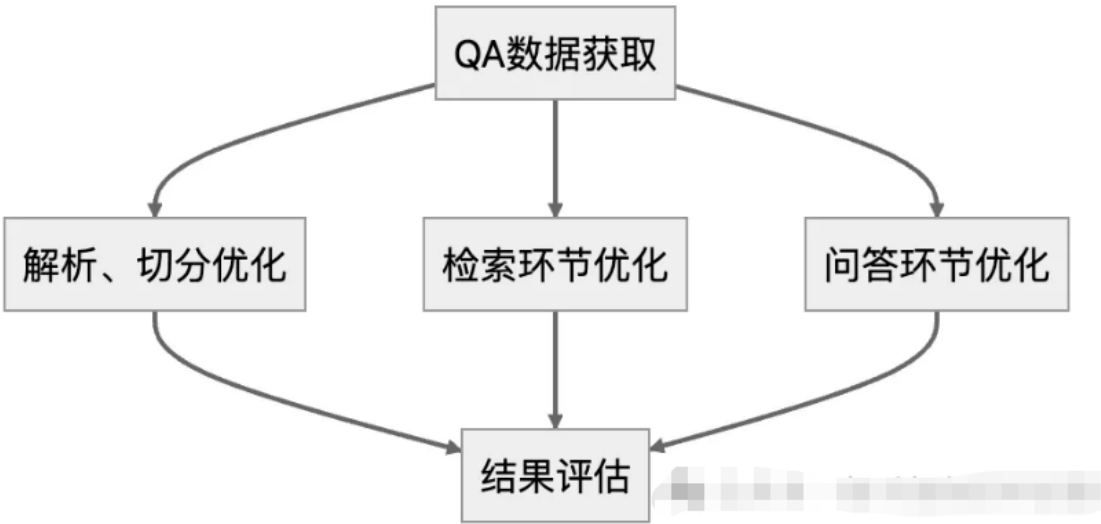
构建一个基础版的RAG是非常简单的，甚至使用扣子、Dify等平台，熟练的情况下都用不了5分钟，即使使用Langchain、LlamaIndex等框架，搭建完整流程，代码也不会超过100行。但基础版的问答效果往往较差。

下面这张图是OpenAI介绍的RAG优化经验，这个准确率当然随不同的数据集会有不同，但基本上优化后的准确率比优化前有显著提升这个基本上是一致的。



问答系统构建完成后，总的流程是先对文档进行解析、切分，然后使用问题检索相关知识片段，最后将问题和知识片段输入LLM，生成答案。

在构建的过程中也是一样的，这三个环节是可以分别独立优化的，如下图所示：



本篇首先专注在如何获取QA数据，所谓的QA数据，就是“问题-回答”数据，理想情况下，如果包含回答所用到的文档片段是更好的。部分系统（如客服系统）是有这方面数据的，但绝大多数情况下是没有的，这时就需要首先构造一批问答数据，这是后续所有环节最重要的一步。

本文所介绍的方法，会使用官方qwen-long模型，对《2024全球经济金融展望报告》这个文档抽取QA，这个模型足够便宜，抽取的结果质量也还不错。QA抽取包含如下3个步骤：

- 短文档片段QA抽取：这部分模拟日常情况下，经常会询问细节性问题的使用场景
- 长文档片段QA抽取：这部分模拟需要综合较多上下文才能回答的使用场景
- QA质量打分：使用LLM再次对抽取的QA进行质量评估，这一步算是借鉴了微软phi-1.5模型Textbooks Are All You Need论文中的方法，就是借助模型对数据质量进行评估

整个过程花费不到1元，结果已经抽取好了，大家可以直接使用。

本文所对应代码已开源，地址在：<https://github.com/Steven-Luo/MasteringRAG/blob/main/00>

二、准备环境

代码在Google Colab环境下进行了测试，正常情况下，安装Anaconda基本上会包含大部分所用到的包，再安装如下包即可：

```
pip install langchain langchain\_community pypdf openai
```

为了便于大家复现，打印所安装的版本：

```
import langchain, langchain\_community, pypdf, openai

for module in (langchain, langchain\_community, pypdf, openai):
    print(f"{module.\_\_name\_\_:<20}{module.\_\_version\_\_}")
```

```
langchain          0.2.8
langchain\_community 0.2.7
pypdf              4.3.0
openai             1.35.14
```

设置API key

```
import os

os.environ\['API\_KEY'\] = '替换为自己的Key'
os.environ\['BASE\_URL'\] = 'https://dashscope.aliyuncs.com/compatible-mode/v1'
```

三、文档解析与切分

```
from langchain\_community.document\_loaders import PyPDFLoader
from langchain.schema import Document
from langchain.text\_splitter import RecursiveCharacterTextSplitter
```

```

import re
from uuid import uuid4

def split_docs(documents, filepath, chunk_size=400, chunk_overlap=40,
separators=['\n\n\n', '\\n\\n'], force_split=False):
    if os.path.exists(filepath) and not force_split:
        print('found cache, restoring...')
        return pickle.load(open(filepath, 'rb'))

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=separators
    )
    split_docs = splitter.split_documents(documents)
    for chunk in split_docs:
        chunk.metadata['uuid'] = str(uuid4())

    pickle.dump(split_docs, open(filepath, 'wb'))

    return split_docs

loader = PyPDFLoader("data/2024全球经济金融展望报告.pdf")
documents = loader.load()

# 由于原documents跨页，导致即使chunk_size设置较大，也没有办法获得更大的文档片段，因此，合
并整个文档后再切分，方便后面构造大上下文QA
pattern = r"^全球经济金融展望报告\\n中国银行研究院 \\d+ 2024年"
merged_docs = \[Document(page_content='\\n'.join(re.sub(pattern, '',
doc.page_content) for doc in documents))\]

splitted_docs = split_docs(documents, os.path.join(output_dir,
'split_docs.pkl'), chunk_size=500, chunk_overlap=50)
splitted_docs_large = split_docs(merged_docs, os.path.join(output_dir,
'split_docs_large.pkl'), chunk_size=1500, chunk_overlap=100)
uuid2doc = {doc.metadata['uuid']: doc.page_content for doc in splitted_docs}

uuid2large_doc = {doc.metadata['uuid']: doc.page_content for doc in
splitted_docs_large}

```

四、QA抽取

既然是构造QA，那最好是保留回答问题时所使用的上下文，方便后续环节的优化。

1、QA抽取Prompt

这一步核心的2个Prompt如下：

```

qa_gen_prompt_tpl = """
我会给你一段文本（<document></document>之间的部分），你需要阅读这段文本，分别针对这段文本生成
8个问题、用户回答这个问题的上下文，和基于上下文对问题的回答。

对问题、上下文、答案的要求：

```

问题要与这段文本相关，不要询问类似“这个问题的答案在哪一章”这样的问题

上下文：上下文必须与原始文本的内容保持一致，不要进行缩写、扩写、改写、摘要、替换词语等

答案：回答请保持完整且简洁，无须重复问题。答案要能够独立回答问题，而不是引用现有的章节、页码等

返回结果以JSON形式组织，格式为`\[{"question": "...", "context": ..., "answer": "..."}, ...\\]`。

如果当前文本主要是目录，或者是一些人名、地址、电子邮箱等没有办法生成有意义的问题时，可以返回`\\]`。

下方是文本：

```
<document>
{{document}}
</document>
```

请生成结果：

"""

```
qa\_gen\_prompt\_tmpl\_large\_context = """
```

我会给你一段文本（`<document></document>`之间的部分），你需要阅读这段文本，分别针对这段文本生成2个问题，和基于这段文本对问题的回答，回答请保持完整，无须重复问题。

尽可能创建一些需要综合`*大段*文本`才能回答的问题，但不要问类似“这一段主要讲了什么内容”这样的问题，答案要能够独立回答问题，而不是引用现有的章节、页码等；不要问具体过于细节的问题，例如“海湾国家的2024年预期经济增长率是多少”，而是尽可能问类似“2024年全球经济的几大趋势是什么”、“受局部中东地区紧张局势影响，可能对全球原物料有哪些影响”。

返回结果以JSON形式组织，格式为`\[{"question": "...", "answer": "..."}, ...\\]`。

如果当前文本主要是目录，或者是一些人名、地址、电子邮箱等没有办法生成有意义的问题时，可以返回`\\]`。

下方是文本：

```
<document>
{{document}}
</document>
```

请生成结果：

"""

2、QA抽取代码

抽取核心代码，此处使用多线程加速抽取，考虑到网络请求异常情况会比较多，因此增加失败重试机制，同时考虑到这是一个耗时操作，并保存中间结果，以确保失败或者再次运行时，已经执行过的部分不会被重复执行：

```
from openai import OpenAI
import time
import random
import threading
import concurrent.futures
from tqdm.auto import tqdm
import json

client = OpenAI(
    api\_key=os.environ\['API\_KEY'\],
    base\_url=os.environ\['BASE\_URL'\])
```

```

)

def build_qa_prompt(prompt_tmpl, text):
    prompt = prompt_tmpl.replace(' ', text).strip()
    return prompt

def chat(prompt, max_retry=3, debug=False, top_p=0.95, temperature=0.85):
    def do_chat(prompt):
        completion = client.chat.completions.create(
            model='qwen-long',
            messages=[
                {"role": "system", "content": "你是一个有用的人工智能助手"},
                {"role": "user", "content": prompt}
            ],
            top_p=top_p,
            temperature=temperature
        )
        return completion.choices[0].message.content

    while max_retry > 0:
        try:
            return do_chat(prompt)
        except Exception as e:
            max_retry -= 1
            sleep_seconds = random.randint(1, 4)
            if debug:
                print(f"{str(e)}, remain retry: {max_retry}, sleeping {sleep_seconds}s {prompt}")
            time.sleep(sleep_seconds)
    return None

def gen_qa(splitted_docs, prompt_tmpl, qa_ckpt_filename):
    qa_ckpt = {}
    if os.path.exists(qa_ckpt_filename):
        qa_ckpt = open(qa_ckpt_filename).readlines()
        qa_ckpt = [json.loads(line.strip()) for line in qa_ckpt if line.strip() != '']
        qa_ckpt = {item['uuid']: item for item in qa_ckpt}
        print(f'found checkpoint, item count: {len(qa_ckpt)}')

    file_lock = threading.Lock()
    # 注意设置并发，如果所使用的API并发限制比较低，调低此值
    max_workers = 4
    with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
        futures = {doc.metadata['uuid']: executor.submit(chat, build_qa_prompt(prompt_tmpl, doc.page_content), 3, True) for doc in splitted_docs if len(doc.page_content.replace('\n', ' ')) >= 150 and doc.metadata['uuid'] not in qa_ckpt}
        for uuid in tqdm(futures):
            future = futures[uuid]
            result = future.result()
            if result is None:
                continue

```

```

        item = {'uuid': uuid, 'raw_resp': result}
        qa\_ckpt\[uuid\] = item

    \# global file\_lock
    file\_lock.acquire()

    try:
        with open(qa\_ckpt\_filename, 'a') as f:
            f.write(json.dumps(item, ensure\_ascii=False) + '\n')
    except Exception as e:
        print(e)
    finally:
        file\_lock.release()

    return qa\_ckpt

\# 短上下文抽取结果
detailed\_qa\_dict = gen\_qa(splitted\_docs, qa\_gen\_prompt\_tmpl,
os.path.join(output\_dir, f"qa\_ckpt\_detailed.jsonl"))
\# 长上下文抽取结果
large\_context\_qa\_dict = gen\_qa(splitted\_docs\_large,
qa\_gen\_prompt\_tmpl\_large\_context, os.path.join(output\_dir,
f"qa\_ckpt\_large\_context.jsonl"))

```

3、抽取样例

```

\\ \\ \\ json
\[
  {
    "question": "海湾六国经济增长受什么影响较大？ ",
    "context": "受国际能源价格走势影响，近年来海湾六国经济增长波动较大。 ",
    "answer": "国际能源价格走势"
  },
  {
    "question": "2022年海湾六国的平均GDP增速是多少？ ",
    "context": "2022年，海湾六国经济增长强劲，实际GDP增速达7.3%。 ",
    "answer": "7.3%"
  },
  {
    "question": "哪个国家在海湾六国中GDP规模最大？ ",
    "context": "沙特占据海湾六国经济体量的半壁江山。2022年，沙特GDP规模达1.1万亿美元",
    "answer": "沙特"
  },
  {
    "question": "2022年阿联酋的GDP规模大约是多少亿美元？ ",
    "context": "阿联酋和卡塔尔GDP规模分别为5075亿和2373亿美元，位列第二和第三位",
    "answer": "5075亿"
  },
  {
    "question": "2022年沙特油气部门在其经济总量中的占比大约是多少？ ",
    "context": "油气部门在海湾六国经济总量中的占比超过40%，在沙特的占比更是接近七成。 ",
    "answer": "接近七成"
  },
]

```

```
{
  "question": "IMF预测2023年海湾六国的平均经济增速是多少？ ",
  "context": "IMF在最新展望报告中将海湾六国2023年经济增速下调0.8个百分点至1.7%。 ",

  "answer": "1.7%"
},
{
  "question": "2023年和2024年阿联酋预计的经济增速是多少？ ",
  "context": "预计2023年和2024年经济增速分别为3.4%和4.0%，为海湾六国中最高。 ",
  "answer": "2023年3.4%，2024年4.0%"
},
{
  "question": "海湾六国经济结构的主要问题是什么？ ",
  "context": "鉴于海湾六国经济结构相对单一，非石油部门虽然为海",
  "answer": "经济结构相对单一"
}
\]
\\\`
```

4、后置处理

从上面的样例可以看出，结果是被 `json...` 包裹的，没有办法直接解析为JSON，使用正则表达式进行后置处理，提取JSON

```
import re
import pandas as pd

def convert2json(text):
    pattern = r'\\\[\.\\*\\\\]'

    text = text.replace('>>>', '')
    try:
        return json.loads(text)
    except:
        match = re.search(pattern, text, re.DOTALL)
        try:
            matched = match.group(0)
            return json.loads(matched)
        except Exception as e:
            print(f"{match}, {str(e)}")

    return \[\]

def build_qa_df(qa_ckpt, uuid2doc_map):
    data = \[\]

    for key, value in tqdm(qa_ckpt.items()):
        text = value['raw_resp']
        qa_list = convert2json(text)

        for item in qa_list:
            question = item.get('question', '').strip()
            answer = item.get('answer', '').strip()
            context = item.get('context', '').strip()
```

```

        if question == '' or answer == '':
            print(qa\_list)
            continue
        data.append({
            'uuid': key,
            'question': question,
            'answer': answer,
            'context': context,
            'doc': uuid2doc\_map[key]
        })
    qa\_df = pd.DataFrame(data)
    return qa\_df

qa\_df = build\_qa\_df(detailed\_qa\_dict, uuid2doc)
qa\_df.drop\_duplicates('question', inplace=True)
qa\_df['qa\_type'] = 'detailed'
large\_context\_qa\_df = build\_qa\_df(large\_context\_qa\_dict,
    uuid2large\_doc)
large\_context\_qa\_df.drop\_duplicates('question', inplace=True)
large\_context\_qa\_df['qa\_type'] = 'large\_context'

qa\_df = pd.concat([qa\_df, large\_context\_qa\_df])

```

五、QA质量检查

这部分就是对qa_df中的问题-回答对，再打一次分，然后过滤低分结果，Prompt如下：

```

qa\_check\_prompt\_tmp1 = """
你是一个优秀的NLP方面的助教，你的任务是帮助检查出题组所出的期末考试的题目。
你需要根据所出的问题（<question></question>之间的部分），以及参考答案（<answer>
</answer>）进行打分，并给出打分理由，分值是一个int类型的值，取值范围为1-5。
好的问题，应该是询问实时、观点等，而不是类似于“这一段描述了什么”，“文本描述了什么”；
好的答案，应该能够直接回答问题，而不是给出在原文中的引用，例如“第3章”等

结果请以JSON形式组织，格式为如下：
{"score": ..., "reason": ...}

问题：
<question>
{{question}}
</question>

答案：
<answer>
{{answer}}
</answer>

请打分：
"""

```

总体又是一个循环，与QA抽取部分非常相似，此处不再粘贴代码，需要的朋友们请访问代码仓库。

1、打分结果样例

2、3分样例

- 问：报告中提到的主要经济体GDP增速变化趋势的图的名称是什么？
- 答：主要经济体GDP增速变化趋势
- 上下文：图2：主要经济体GDP增速变化趋势（%）

3、2分样例

- 问：消费者借贷能力和意愿受到什么因素的影响？
- 答：美国家庭债务余额拖欠率回升至3%
- 上下文：美国家庭债务余额拖欠率回升至3%，消费者借贷能力和意愿将有所下降。

可以看出，低分问答对，质量确实相对较低

4、最终数据集构建

这部分首先保留4分及以上的问答对，然后随机挑选100条数据作为后续的测试集。至此，准备工作完成。

```
hq\_qa\_df = qa\_df[qa\_df['score'] >= 4]
test\_q = hq\_qa\_df.sample(100, replace=False)['question'].values.tolist()
hq\_qa\_df['dataset'] = 'train'
hq\_qa\_df.loc[hq\_qa\_df['question'].isin(test\_q), 'dataset'] = 'test'

hq\_qa\_df.to\_excel(os.path.join(output\_dir, 'question\_answer.xlsx'),
index=False)
```